Network Adminstration and Security

UNIVERSITY STUDY NOTES

These notes cover the full Advanced Computer Networks curriculum, building on I2208-level prerequisites. Topics include the Network Layer (routing, NAT), Transport Layer (UDP, TCP), Application Layer (HTTP, FTP, Mail, DNS), and Network Security (cryptography, digital signatures, security protocols). Each topic is explained with definitions, mechanisms, packet structures, and practical examples.

Introduction & Network Basics Recall

PREREQUISITE REFRESHER

This course assumes familiarity with I2208-level networking. The following refresher covers essential concepts needed for the advanced material.

The OSI & TCP/IP Models

The **OSI model** has 7 layers: Physical, Data Link, Network, Transport, Session, Presentation, Application. The **TCP/IP model** condenses these into 4 layers: Link, Internet, Transport, Application. This course focuses on layers 3–5 and the Application layer.

Encapsulation & PDUs

Each layer wraps data from the layer above, adding its own header (and sometimes trailer). The data unit at each layer has a distinct name: **Frame** (Data Link), **Packet** (Network), **Segment** (Transport), **Data** (Application).

Addressing

MAC addresses (48-bit, Data Link layer) identify devices on a local network. **IP addresses** (32-bit IPv4 or 128-bit IPv6, Network layer) identify devices globally. ARP maps IP addresses to MAC addresses on a local segment.

IP Packet Structure — Recall

An IPv4 header is typically 20 bytes (without options). Key fields:

Field	Size	Purpose
Version	4 bits	IP version (4 or 6)
IHL	4 bits	Header length in 32-bit words
Total Length	16 bits	Entire packet size in bytes (max 65,535)
TTL	8 bits	Decremented at each hop; packet discarded at zero
Protocol	8 bits	Upper-layer protocol (6=TCP, 17=UDP, 1=ICMP)
Source IP	32 bits	Sender's IP address
Destination IP	32 bits	Receiver's IP address

Network Layer: Routing

The Network Layer (Layer 3) is responsible for **logical addressing** and **routing** — determining the path a packet takes from source to destination across potentially many networks. Routers use **routing tables** to make forwarding decisions.

Static Routing

What It Is

Routes are manually configured by a network administrator. The routing table does not change unless an administrator modifies it.

- Simple to configure on small networks
- No routing protocol overhead
- Predictable, deterministic paths
- Does not adapt to failures automatically

How It Works

An administrator enters entries such as:

```
ip route 192.168.2.0 255.255.255.0 10.0.0.2
```

This says: "To reach network 192.168.2.0/24, forward packets to next-hop router 10.0.0.2." The router matches the destination IP against its table using **longest prefix match** — the most specific matching route wins.

Dynamic Routing Algorithms

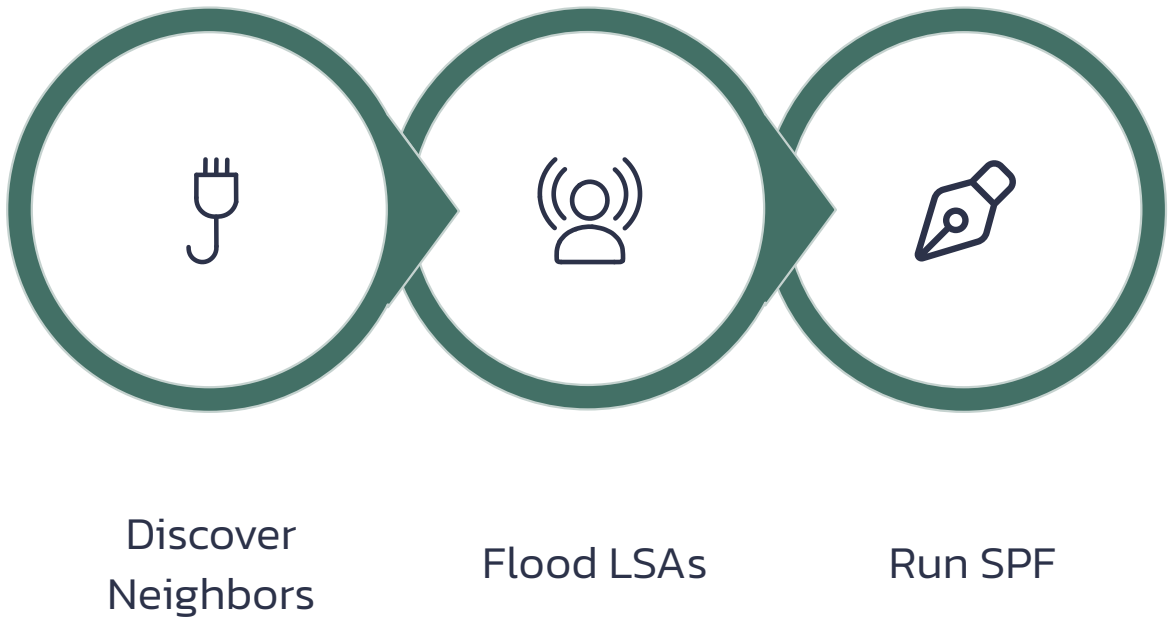
Dynamic routing protocols automatically discover networks and adapt to topology changes. Two fundamental algorithmic families exist:

Distance Vector (Bellman–Ford)

Each router knows only the **distance** (metric) and **vector** (next hop) to each destination. Routers share their entire routing table with **direct neighbors** periodically. Slow to converge; susceptible to **count-to-infinity** problem. Examples: RIP, IGRP.

Link State (Dijkstra)

Each router builds a complete **topology map** of the network by flooding **Link State Advertisements (LSAs)** to all routers. Each router independently runs **Dijkstra's Shortest Path First (SPF)** algorithm. Fast convergence; more CPU and memory intensive. Examples: OSPF, IS-IS.



Link-state routing operates in three phases: first, each router discovers its directly connected neighbors and measures the cost of each link; second, it floods Link State Advertisements (LSAs) throughout the network so every router receives a complete topology map; third, each router independently runs Dijkstra's Shortest Path First (SPF) algorithm to compute the shortest path to every destination and populate its routing table.

Dynamic Routing Protocols & NAT

Major Dynamic Routing Protocols

Protocol	Type	Metric	Algorithm	Key Characteristics
RIP	Distance Vector	Hop count (max 15)	Bellman-Ford	Simple; sends full table every 30s; slow convergence
EIGRP	Advanced Distance Vector	Bandwidth + Delay (composite)	DUAL	Cisco proprietary; fast convergence; partial updates
OSPF	Link State	Cost (based on bandwidth)	Dijkstra SPF	Open standard; areas for scalability; fast convergence
BGP	Path Vector	Policy + path attributes	Path Vector	Internet's inter-domain protocol; policy-based routing

NAT — Network Address Translation

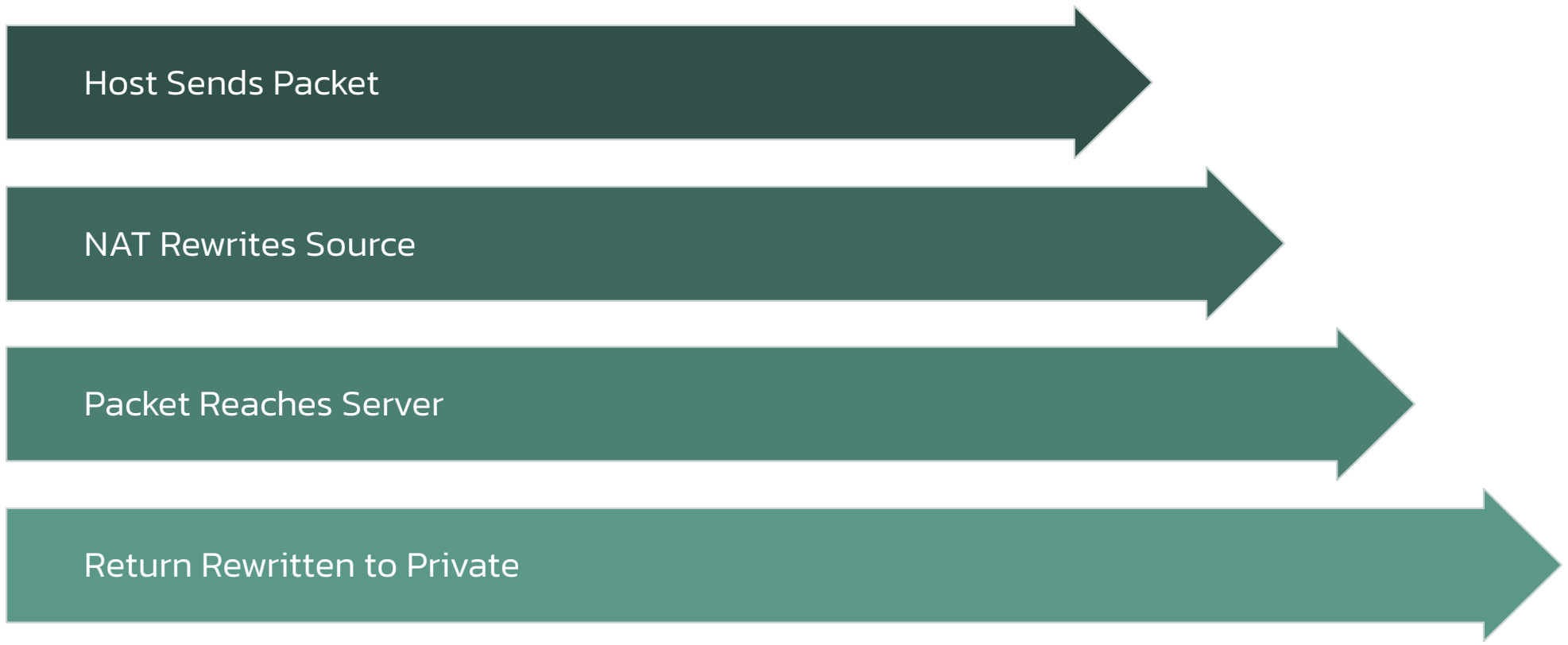
Why NAT Exists

IPv4 has only ~4.3 billion addresses, exhausted long ago. NAT allows an entire private network to share a single (or a few) public IP address(es). Private address ranges (RFC 1918) include 10.0.0.0/8, 172.16.0.0/12, and 192.168.0.0/16.

How NAT Works

A NAT router maintains a **translation table** mapping private IP:port pairs to public IP:port pairs. When an internal host sends a packet outward, the router rewrites the **source IP and port**. Return traffic is matched against the table and the original private address is restored.

- **Static NAT:** One-to-one fixed mapping
- **Dynamic NAT:** Pool of public addresses
- **PAT (NAT Overload):** Many private hosts share one public IP via unique port numbers — most common in practice



This flow illustrates how PAT (NAT overload) enables multiple private hosts to share a single public IP by tracking unique port mappings for each active connection.

Transport Layer: Overview & UDP

The Transport Layer (Layer 4) provides **end-to-end communication** between processes on different hosts. It uses **port numbers** (0–65535) to multiplex multiple applications on a single host. Well-known ports: 0–1023; registered ports: 1024–49151.

Core Functions

- Process-to-process delivery via port numbers
- Multiplexing and demultiplexing
- Error detection (checksum)
- Optional reliability, flow control, congestion control

Two Transport Protocols

UDP — connectionless, minimal overhead, no guarantees. Used for DNS, streaming, VoIP, online gaming.

TCP — connection-oriented, reliable, ordered delivery with flow and congestion control. Used for HTTP, FTP, email, SSH.

UDP — User Datagram Protocol

UDP Header (8 bytes)

Field	Size	Purpose
Source Port	16 bits	Sender's port
Destination Port	16 bits	Receiver's port
Length	16 bits	Header + data in bytes
Checksum	16 bits	Error detection (optional in IPv4)

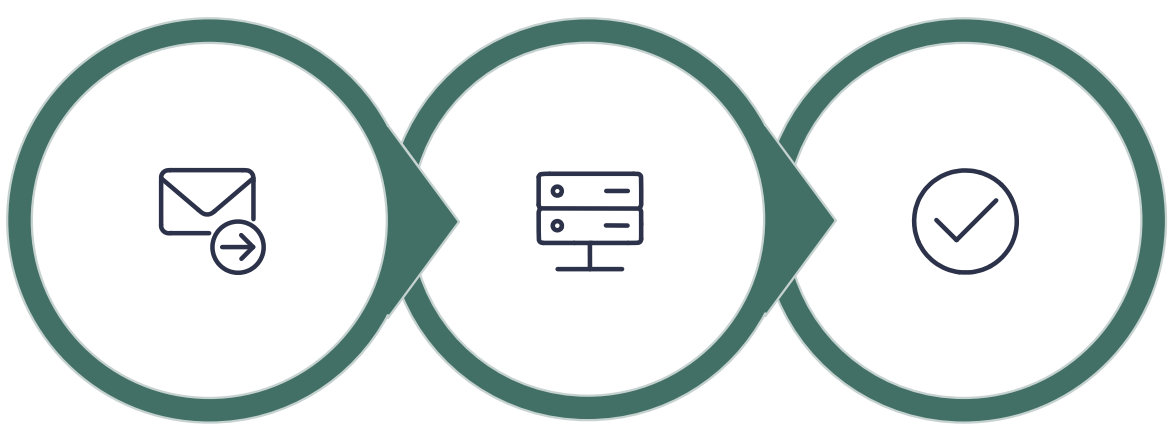
UDP Characteristics

- **Connectionless:** No handshake; datagrams sent immediately
- **No reliability:** Packets may be lost, duplicated, or arrive out of order
- **No flow or congestion control:** Sender transmits at application rate
- **Low overhead:** 8-byte header vs. TCP's 20+ bytes
- **Low latency:** Ideal for real-time applications

TCP: Connection Management

TCP (Transmission Control Protocol) provides **reliable, ordered, error-checked** delivery of a byte stream between two endpoints. Before data transfer, TCP establishes a connection; after transfer, it terminates it cleanly.

The Three-Way Handshake



Client SYN | Server SYN-ACK | Client ACK

Each side chooses an **Initial Sequence Number (ISN)**. The SYN and ACK flags consume one sequence number each. Both sides learn the other's ISN and confirm bidirectional connectivity before transmitting data.

TCP Header — Key Fields

Field	Size	Purpose
Source/Dest Port	16 bits each	Identify sending and receiving processes
Sequence Number	32 bits	Byte number of first data byte in segment (or ISN in SYN)
Acknowledgment Number	32 bits	Next expected byte from the other side (valid when ACK=1)
Flags	6 bits	SYN, ACK, FIN, RST, PSH, URG — control connection state
Window Size	16 bits	Receiver's available buffer space (flow control)
Checksum	16 bits	Error detection over header, data, and pseudo-header

Connection Termination — Four-Way Handshake

Either side can initiate closure by sending **FIN**. Because TCP is full-duplex, each direction closes independently:

1. Initiator sends **FIN** (closes its sending side)
2. Receiver sends **ACK** (may still send data)
3. Receiver sends its own **FIN** when ready
4. Initiator sends **ACK**

The initiator enters **TIME_WAIT** for 2×MSL (Maximum Segment Lifetime) to absorb any delayed duplicate segments before the connection fully closes.

TCP: Flow Control & Congestion Control

Flow Control — Sliding Window

Flow control prevents a fast sender from overwhelming a slow receiver. TCP uses a **sliding window** mechanism: the receiver advertises its available buffer space in the **Window Size** field of every ACK.

How the Sliding Window Works

The sender maintains a **send window** bounded by:

- **Last Acknowledged Byte** (left edge)
- **Last Acknowledged + Window Size** (right edge)

The sender may transmit any unacknowledged bytes within this window. As ACKs arrive, the window **slides forward**. If the receiver's buffer fills, it advertises a **window size of zero**, pausing the sender until buffer space is available.

Key Concepts

- **Receiver Window (rwnd):** Flow control limit set by receiver
- **Congestion Window (cwnd):** Congestion control limit set by sender
- **Effective Window:** min(rwnd, cwnd) — actual sending limit
- **Nagle's Algorithm:** Buffers small segments to reduce packet count
- **Delayed ACK:** Receiver may wait up to 500ms to piggyback ACK on outgoing data

Congestion Control

Congestion control prevents a sender from overwhelming the **network** (not just the receiver). TCP infers congestion from **packet loss** (timeout or 3 duplicate ACKs) and adjusts **cwnd** accordingly.

1

Slow Start

cwnd starts at 1 MSS and **doubles every RTT** (exponential growth). Continues until cwnd reaches **ssthresh** (slow-start threshold) or loss occurs.

2

Congestion Avoidance

After ssthresh, cwnd grows **linearly** (+1 MSS per RTT). More conservative than Slow Start; used when network capacity is approached.

3

Fast Retransmit

On receiving **3 duplicate ACKs**, sender retransmits the missing segment immediately — without waiting for a timeout. Indicates isolated packet loss.

4

Fast Recovery

After Fast Retransmit, cwnd is halved and ssthresh is set to the new cwnd. TCP enters Congestion Avoidance directly — *not* Slow Start. (Reno and later variants)

Application Layer Protocols

The Application Layer provides network services directly to end-user applications. Each protocol defines message formats, port numbers, and client-server interaction patterns.



HTTP — Hypertext Transfer Protocol

Ports: 80 (HTTP), 443 (HTTPS). **Model:** Client sends request, server returns response. Stateless by design (each request independent). Methods: GET, POST, PUT, DELETE, HEAD. Status codes: 2xx (success), 3xx (redirect), 4xx (client error), 5xx (server error). HTTPS adds TLS encryption below HTTP.



FTP — File Transfer Protocol

Ports: 21 (control), 20 (active data). Uses **two separate TCP connections**: control (commands) and data (file content). **Active mode:** server initiates data connection to client. **Passive mode:** client initiates both connections (firewall-friendly). Supports authentication, directory navigation, and binary/ASCII transfer modes.



Mail Protocols

SMTP (port 25/587): Pushes mail from client to server and between servers. **POP3** (port 110): Downloads mail to client, typically deletes from server. **IMAP** (port 143/993): Manages mail on server; supports folders, multiple clients, and partial downloads. IMAP is preferred for modern multi-device access.



DNS — Domain Name System

Port: 53 (UDP for queries, TCP for zone transfers). Translates domain names to IP addresses. **Hierarchical:** Root → TLD (.com, .org) → Authoritative servers. **Query types:** Iterative (server refers client to next server) and Recursive (server resolves fully on client's behalf). Records: A, AAAA, CNAME, MX, NS, TXT.

Introduction to Network Security

Network security ensures that communication over potentially hostile networks remains **confidential, authentic, and intact**. Security is built from cryptographic primitives combined into protocols.

Security Services

Confidentiality

Only authorized parties can read the data. Achieved via **encryption**.

Integrity

Data has not been altered in transit. Achieved via **hashing** and MACs.

Authentication

Verifying the identity of a user, host, or service. Achieved via passwords, certificates, or digital signatures.

Non-Repudiation

Sender cannot deny having sent a message. Achieved via **digital signatures**.

Cryptography Fundamentals

Symmetric Cryptography

Same key for encryption and decryption. Fast and efficient for bulk data.

- **AES** (128/192/256-bit) — current standard
- **DES / 3DES** — legacy, largely deprecated
- Challenge: secure key distribution at scale

Asymmetric Cryptography

Uses a **public key** (encrypt/verify) and **private key** (decrypt/sign). Solves key distribution but is slower.

- **RSA** — encryption and digital signatures
- **ECC** — smaller keys, equivalent security
- **Diffie-Hellman** — key exchange only

In practice, asymmetric crypto establishes a session key; symmetric crypto encrypts the data.

Hashing & Digital Signatures

A **cryptographic hash** (e.g., SHA-256) maps any input to a fixed-length digest. Properties: deterministic, fast, one-way, collision-resistant. Used for integrity checks and password storage.

A **digital signature** combines hashing and asymmetric encryption: the sender hashes the message, encrypts the hash with their **private key**, and attaches it. The receiver decrypts with the sender's **public key** and compares hashes — verifying both integrity and authenticity.

Principles of Network Security Protocols

TLS / SSL

Secures HTTP (HTTPS), SMTP, IMAP, and others. Uses asymmetric crypto for authentication and key exchange, then symmetric crypto for data. Provides confidentiality, integrity, and server (optionally client) authentication.

IPSec

Secures IP packets at the Network Layer. Operates in **Transport mode** (payload only) or **Tunnel mode** (entire packet). Uses AH (authentication) and ESP (encryption + authentication). Foundation of VPNs.

Defense in Depth

No single mechanism is sufficient. Effective security layers multiple controls: firewalls, encryption, authentication, access control, and monitoring — so that compromise of one layer does not compromise the whole system.